

File Management in Ubuntu Server 24.04

A Complete System Administration Reference
Commands, Permissions, Storage, Archiving & Automation

About This Guide

This document covers the essential file management techniques for Ubuntu Server 24.04 LTS (Noble Numbat). It is intended for system administrators, DevOps engineers, and developers who work with Ubuntu Server environments and need a reliable command reference for daily operations.

1. Linux Filesystem Structure

Ubuntu Server follows the Filesystem Hierarchy Standard (FHS), which defines the directory structure and directory contents in Linux distributions. Understanding this hierarchy is fundamental to effective file management.

Command	Description
/	Root directory — the top of the filesystem tree
/bin	Essential user binaries (ls, cp, mv, etc.)
/sbin	Essential system binaries (fdisk, ifconfig, etc.)
/etc	System-wide configuration files
/home	User home directories
/root	Home directory for the root user
/var	Variable data: logs, mail, spool files
/tmp	Temporary files (cleared on reboot)
/usr	Secondary hierarchy for read-only user data
/opt	Optional third-party software packages
/proc	Virtual filesystem exposing kernel/process info
/sys	Virtual filesystem for hardware and kernel data
/dev	Device files (disks, terminals, etc.)
/mnt	Mount point for temporary mounts

/media	Auto-mount point for removable media
/srv	Data for services provided by the system
/boot	Boot loader files and kernel images
/lib	Shared libraries for /bin and /sbin

2. Essential File & Directory Commands

2.1 Navigating the Filesystem

These commands allow you to move through and inspect directories.

Command	Description
pwd	Print the current working directory
ls	List directory contents
ls -la	Long listing including hidden files
ls -lh	Long listing with human-readable sizes
cd /path	Change to an absolute directory
cd ~	Change to the home directory
cd ..	Go up one directory level
cd -	Return to the previous directory
tree	Display directory structure as a tree
tree -L 2	Display tree limited to 2 levels deep

2.2 Creating Files and Directories

```
# Create an empty file
touch filename.txt

# Create a file with content
echo 'Hello, World!' > filename.txt

# Create a directory
mkdir my_directory

# Create nested directories at once
mkdir -p /var/www/html/project/assets

# Create multiple directories
```

```
mkdir dir1 dir2 dir3
```

2.3 Copying, Moving & Renaming

Command	Description
<code>cp file1 file2</code>	Copy a file
<code>cp -r dir1 dir2</code>	Copy a directory recursively
<code>cp -p file1 file2</code>	Copy preserving timestamps and permissions
<code>cp -a src dst</code>	Archive copy (preserves all attributes)
<code>mv file1 file2</code>	Rename a file or move it
<code>mv file /new/path/</code>	Move a file to another directory
<code>mv -i src dst</code>	Interactive move (prompt before overwrite)
<code>mv -n src dst</code>	Move without overwriting existing files

2.4 Deleting Files and Directories

Warning

The `rm` command is irreversible on most Ubuntu server configurations. There is no Recycle Bin. Always double-check your command before pressing Enter, especially with wildcards or the `-rf` flags.

```
# Remove a file
rm filename.txt

# Prompt before each removal
rm -i filename.txt

# Remove an empty directory
rmdir my_directory

# Remove a directory and all its contents
rm -rf /path/to/directory

# Remove all .log files in current directory
rm *.log
```

2.5 Viewing File Contents

Command	Description
<code>cat file.txt</code>	Display entire file contents

<code>less file.txt</code>	Page through a file (q to quit)
<code>more file.txt</code>	Page through a file (older tool)
<code>head file.txt</code>	Display first 10 lines
<code>head -n 20 file.txt</code>	Display first 20 lines
<code>tail file.txt</code>	Display last 10 lines
<code>tail -n 50 file.txt</code>	Display last 50 lines
<code>tail -f /var/log/syslog</code>	Follow a file in real-time
<code>wc -l file.txt</code>	Count lines in a file
<code>wc -w file.txt</code>	Count words in a file

3. File Permissions & Ownership

3.1 Understanding Permission Notation

Every file and directory has three permission classes: owner (u), group (g), and others (o). Each class can have read (r=4), write (w=2), and execute (x=1) permissions.

```
# Example output of ls -la
-rwxr-xr-- 1 alice developers 4096 May 10 09:00 script.sh
drwxr-x--- 2 alice developers 4096 May 10 09:00 project/

# Format: [type][owner][group][others]
# - = regular file
# d = directory
# l = symbolic link
# rwx = read, write, execute
```

3.2 Changing Permissions with chmod

Command	Description
<code>chmod 755 file</code>	rw-r-xr-x (owner all, group/others read+exec)
<code>chmod 644 file</code>	rw-r--r-- (owner read/write, others read)
<code>chmod 600 file</code>	rw----- (owner only — private files)
<code>chmod 777 file</code>	rw-rwxrwx (everyone full access — avoid!)
<code>chmod u+x file</code>	Add execute for owner
<code>chmod g-w file</code>	Remove write from group
<code>chmod o=r file</code>	Set others to read only

<code>chmod -R 755 dir/</code>	Apply permissions recursively
<code>chmod a+x file</code>	Add execute for all users

Best Practice: Principle of Least Privilege

Always assign the minimum permissions necessary. Use 600 for sensitive config files, 644 for public files, 750 for private scripts, and 755 for public executables. Avoid 777 in production environments.

3.3 Changing Ownership with chown and chgrp

```
# Change file owner
sudo chown alice file.txt

# Change file owner and group
sudo chown alice:developers file.txt

# Change only the group
sudo chgrp developers file.txt

# Recursively change ownership of a directory
sudo chown -R www-data:www-data /var/www/html

# Change ownership to match another file
sudo chown --reference=ref_file.txt target_file.txt
```

3.4 Special Permissions

Command	Description
<code>chmod u+s file</code>	Set SUID bit (run as file owner)
<code>chmod g+s dir/</code>	Set SGID bit (new files inherit group)
<code>chmod +t dir/</code>	Set sticky bit (only owner can delete files)
<code>chmod 4755 file</code>	SUID + standard 755
<code>chmod 2775 dir/</code>	SGID + standard 775
<code>chmod 1777 dir/</code>	Sticky + full permissions (like /tmp)

3.5 Access Control Lists (ACLs)

ACLs allow fine-grained permissions beyond the standard owner/group/others model. Install with: `sudo apt install acl`

```
# View ACLs on a file
```

```
getfacl file.txt

# Grant user bob read/write access
sudo setfacl -m u:bob:rw file.txt

# Grant group devops read access
sudo setfacl -m g:devops:r file.txt

# Remove ACL for user bob
sudo setfacl -x u:bob file.txt

# Remove all ACLs
sudo setfacl -b file.txt

# Set default ACL for new files in directory
sudo setfacl -d -m u:bob:rw /shared/project
```

4. Searching & Finding Files

4.1 The find Command

The find command is the most powerful tool for locating files based on a wide variety of criteria.

```
# Find files by name (case-sensitive)
find /var/log -name 'syslog'

# Find files by name (case-insensitive)
find / -iname '*.conf' 2>/dev/null

# Find files modified in the last 7 days
find /home -mtime -7

# Find files larger than 100MB
find / -size +100M -type f

# Find files with specific permissions
find / -perm 777 -type f

# Find and delete old log files
find /var/log -name '*.gz' -mtime +30 -delete

# Find and execute a command on results
find /tmp -name '*.tmp' -exec rm -f {} \;

# Find files owned by a user
find /home -user alice -type f
```

4.2 locate and which

Command	Description
<code>locate filename</code>	Fast file search using a pre-built database
<code>sudo updatedb</code>	Update the locate database
<code>locate -i filename</code>	Case-insensitive search
<code>locate '*.conf' grep nginx</code>	Locate + filter with grep
<code>which python3</code>	Find the path of an executable
<code>whereis python3</code>	Find binary, source, and man pages
<code>type ls</code>	Show how a command is interpreted

4.3 Searching File Contents with grep

```
# Basic pattern search
grep 'error' /var/log/syslog

# Case-insensitive search
grep -i 'warning' /var/log/auth.log

# Search recursively in a directory
grep -r 'database_password' /etc/

# Show line numbers
grep -n 'FAILED' /var/log/auth.log

# Invert match (exclude lines)
grep -v '#' /etc/nginx/nginx.conf

# Count matching lines
grep -c 'error' /var/log/syslog

# Use extended regex
grep -E 'error|warning|critical' /var/log/syslog

# Show context (2 lines before and after)
grep -C 2 'segfault' /var/log/kern.log
```

5. Links: Hard & Symbolic

5.1 Hard Links

A hard link is an additional directory entry pointing to the same inode (data) as the original file. Deleting the original does not affect the hard link.

```
# Create a hard link
ln original.txt hardlink.txt

# Verify same inode (first column)
ls -li original.txt hardlink.txt
```

5.2 Symbolic (Soft) Links

A symbolic link is a pointer to another file or directory path. It can cross filesystem boundaries and can point to directories. If the target is deleted, the symlink becomes broken.

```
# Create a symbolic link
ln -s /path/to/original /path/to/link

# Common use case: link config files
sudo ln -s /etc/nginx/sites-available/mysite /etc/nginx/sites-enabled/mysite

# Link a binary to /usr/local/bin
sudo ln -s /opt/myapp/bin/myapp /usr/local/bin/myapp

# Find broken symlinks
find /etc -xtype l

# Remove a symlink (do NOT use rm -rf on a directory symlink!)
rm /path/to/symlink
```

6. Archiving & Compression

6.1 tar — Tape Archive

Command	Description
<code>tar -cvf archive.tar dir/</code>	Create a tar archive
<code>tar -xvf archive.tar</code>	Extract a tar archive
<code>tar -tvf archive.tar</code>	List contents without extracting
<code>tar -cvzf archive.tar.gz dir/</code>	Create gzip-compressed archive
<code>tar -xvzf archive.tar.gz</code>	Extract gzip-compressed archive
<code>tar -cvjf archive.tar.bz2 dir/</code>	Create bzip2-compressed archive
<code>tar -cvJf archive.tar.xz dir/</code>	Create xz-compressed archive (best ratio)
<code>tar -xvf file.tar -C /dest/</code>	Extract to specific directory
<code>tar --exclude='*.log' -cvzf a.tgz dir/</code>	Exclude files matching pattern

6.2 gzip, bzip2, and xz

```
# gzip compression (fast, moderate ratio)
gzip largefile.txt           # Creates largefile.txt.gz
gunzip largefile.txt.gz      # Decompress

# bzip2 (slower, better ratio than gzip)
bzip2 largefile.txt          # Creates largefile.txt.bz2
bunzip2 largefile.txt.bz2

# xz (slowest, best compression ratio)
xz -9 largefile.txt          # Creates largefile.txt.xz (max compression)
xz -d largefile.txt.xz       # Decompress

# zip / unzip
zip -r archive.zip directory/
unzip archive.zip -d /destination
```

7. Disk Usage & Storage Management

7.1 Monitoring Disk Space

Command	Description
<code>df -h</code>	Disk free space for all filesystems
<code>df -h /</code>	Disk space for the root filesystem
<code>df -i</code>	Show inode usage
<code>du -sh /var</code>	Total size of a directory
<code>du -sh /*</code>	Size of every top-level directory
<code>du -ah /var/log sort -rh head -20</code>	Top 20 largest files/dirs in /var/log
<code>ncdu /</code>	Interactive ncurses disk usage viewer

```
# Install ncdu (highly recommended tool)
sudo apt install ncdu

# Find the top 10 largest files on the entire system
find / -type f -printf '%s %p\n' 2>/dev/null | sort -rn | head -10

# Find directories using more than 1GB
du -h --max-depth=3 / 2>/dev/null | grep '^[0-9.]*G'
```

7.2 Managing Partitions with fdisk and parted

Caution

Partition management directly modifies disk structure. Errors can result in data loss. Always back up data before making partition changes. Use fdisk for MBR disks and parted or gdisk for GPT disks.

```
# List all disks and partitions
sudo fdisk -l

# Interactive partition manager for /dev/sdb
sudo fdisk /dev/sdb

# Use parted for GPT/large disks
sudo parted /dev/sdb

# Print partition table
(parted) print

# Create a new partition
(parted) mkpart primary ext4 1MiB 100%
```

7.3 Formatting & Mounting Filesystems

```
# Format a partition as ext4
sudo mkfs.ext4 /dev/sdb1

# Format as XFS (common for large filesystems)
sudo mkfs.xfs /dev/sdb1

# Create a mount point
sudo mkdir -p /mnt/data

# Temporary mount
sudo mount /dev/sdb1 /mnt/data

# Verify mount
df -h /mnt/data

# Unmount
sudo umount /mnt/data

# Permanent mount: add entry to /etc/fstab
# UUID=xxxx-xxxx /mnt/data ext4 defaults 0 2
sudo blkid /dev/sdb1 # Get UUID
sudo nano /etc/fstab # Add entry
sudo mount -a        # Test fstab entry
```

8. Text File Processing

8.1 sed — Stream Editor

```
# Replace first occurrence per line
sed 's/old/new/' file.txt

# Replace all occurrences (global)
sed 's/old/new/g' file.txt

# In-place edit (modifies the file)
sed -i 's/old/new/g' file.txt

# Delete lines containing a pattern
sed '/pattern/d' file.txt

# Print specific line number (e.g., line 5)
sed -n '5p' file.txt

# Print lines 10-20
sed -n '10,20p' file.txt
```

8.2 awk — Text Processing

```
# Print second column (space-delimited)
awk '{print $2}' file.txt

# Print lines where 3rd column > 100
awk '$3 > 100' file.txt

# Sum values in column 1
awk '{sum += $1} END {print sum}' file.txt

# Use custom delimiter (CSV)
awk -F',' '{print $1, $3}' data.csv

# Print lines between patterns
awk '/START/,/END/' file.txt
```

9. File Backup Strategies

9.1 rsync — Fast Incremental Backup

rsync is the standard tool for local and remote backup. It only transfers changed files, making it efficient for large datasets.

Command	Description
<code>rsync -av src/ dst/</code>	Sync directories, verbose, archive mode
<code>rsync -avz src/ user@host:dst/</code>	Sync to remote server with compression
<code>rsync -av --delete src/ dst/</code>	Mirror (delete files not in source)
<code>rsync -av --exclude='*.log' src/ dst/</code>	Exclude log files from sync
<code>rsync -avz --progress src/ dst/</code>	Show progress during transfer
<code>rsync -n -av src/ dst/</code>	Dry run (preview without making changes)

```
# Full server backup example
rsync -avz --exclude='/proc' --exclude='/sys' \
  --exclude='/dev' --exclude='/tmp' \
  / backup@192.168.1.10:/backups/server1/

# Backup with timestamp
rsync -av /var/www/ /backups/www-$(date +%Y%m%d)/
```

9.2 Automated Backups with cron

```
# Edit the root crontab
sudo crontab -e

# Cron syntax:
# minute hour day month weekday command

# Daily backup at 2:00 AM
0 2 * * * rsync -az /var/www/ /backups/www/

# Weekly backup every Sunday at 3:30 AM
30 3 * * 0 tar -czf /backups/full-$(date +%Y%m%d).tar.gz /etc /home /var

# Delete backups older than 30 days
0 4 * * * find /backups -name '*.tar.gz' -mtime +30 -delete
```

10. Useful File Management Tips

10.1 File Attributes with lsattr/chattr

Command	Description
<code>lsattr file.txt</code>	List file attributes
<code>sudo chattr +i file.txt</code>	Make file immutable (cannot be deleted or modified)

<code>sudo chattr -i file.txt</code>	Remove immutable flag
<code>sudo chattr +a file.txt</code>	Append-only (useful for log files)
<code>sudo chattr -R +i /etc/</code>	Make entire directory immutable

10.2 Checking File Types

```
# Determine file type
file document.pdf
file /usr/bin/python3
file archive.tar.gz

# Output: document.pdf: PDF document, version 1.7
```

10.3 Sorting and Redirecting Output

```
# Sort file contents alphabetically
sort file.txt

# Sort numerically
sort -n numbers.txt

# Sort and remove duplicates
sort -u file.txt

# Redirect stdout to file (overwrite)
ls -la > listing.txt

# Append stdout to file
echo 'new entry' >> listing.txt

# Redirect stderr to file
command 2> errors.log

# Redirect both stdout and stderr
command > output.log 2>&1
```

10.4 Quick Reference: Permission Octal Chart

Command	Description
<code>777</code>	<code>rw-rw-rw-</code> — Full access for all (use carefully)
<code>755</code>	<code>rw-r-xr-x</code> — Standard for executables/directories
<code>750</code>	<code>rw-r-x---</code> — Scripts accessible by group

700	rw----- — Private executables (owner only)
664	rw-rw-r-- — Collaborative files (group writable)
644	rw-r--r-- — Standard for public files
640	rw-r----- — Config files readable by group
600	rw----- — Private files (SSH keys, etc.)
400	r----- — Read-only, maximum restriction

Additional Resources

For more information, refer to the official Ubuntu Server documentation at ubuntu.com/server/docs, the man pages (`man <command>`), and the Ubuntu Community Hub at discourse.ubuntu.com. Use 'man hier' to learn more about the Linux filesystem hierarchy.